MyScoutee
MYSCOUTEE TECHNICAL DOCUMENTATION

AUTHENTICATION LABORATORY

MyScoutee 2FA Technical Guide

A standalone Go sandbox for TOTP, HOTP, WebAuthn passkeys, recovery codes, and optional OIDC and SAML identity flows.

TRUST BOUNDARY

The factor API assumes a trusted application has already identified the user. Its bearer token is a server secret. The optional browser and Keycloak flows are a local laboratory; they do not create MyScoutee sessions.

Document ID MSC-2FA-001	**Version** 1.5.0
Software myscoutee-2fa Go service	**Status** DEVELOPMENT SANDBOX
Date 28 October 2026	**Audience** Developers and operators

Document control

Purpose	Describe the current runnable sandbox and the boundary for a later MyScoutee backend integration.
Scope	TOTP, HOTP, WebAuthn, recovery codes, optional Keycloak OIDC/SAML flows, HTTP API, Docker, storage and security.
Sources	The `myscoutee-2fa` repository: `cmd/`, `internal/mfa/`, Compose files and `README.md`.
Integration state	The service runs independently. Current MyScoutee login and session handling do not call it.

Responsibility of the integrating backend

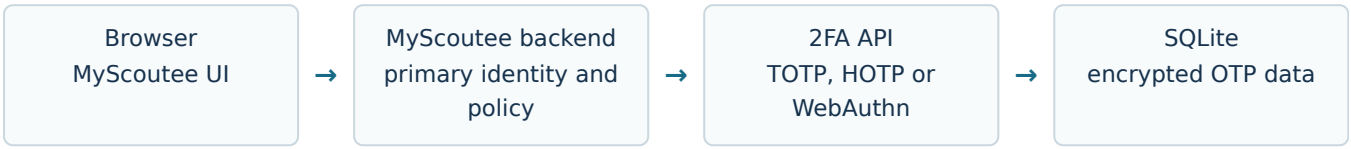
Resolve and authorize the internal user ID, decide when a factor is required, and create or authorize the application session only after its own policy succeeds. The 2FA service verifies a proof; it does not make the MyScoutee login decision.

Contents

1. Architecture and responsibilities
2. Local start and trial
3. Factor lifecycle
4. Why generated codes protect an account
5. Passkey methodology
6. HTTP API
7. Docker, nginx and identity lab
8. Data and security boundaries
9. MyScoutee integration cases
10. Verification and current limits
11. Appendix A: technology coverage
12. Appendix B: standards and sources

1 Architecture and responsibilities

`myscoutee-2fa` is an independent HTTP service. Trusted server callers manage and verify a user's second factors. A small browser page exercises the same endpoints locally.



The development Compose publishes port 18083 only on host loopback. In a later deployment, the Go service can remain on a private network; nginx can supply the HTTPS origin required by browser WebAuthn. This is an integration design, not a deployed MyScoutee connection.

Component	Responsibility
Calling backend	Verified user ID, authorization, factor policy and application session.
2FA API	Factor enrollment, challenges, proof verification and factor state.
Browser	WebAuthn ceremony and OTP or recovery code entry.
SQLite volume	Factor records, public passkey credentials, ceremony state and recovery hashes.

An optional Keycloak container supplies OIDC and SAML identity flows. Those protocols demonstrate possible upstream identity providers. Their result is separate from the factor API.

2 Local start and trial

1. Copy `.env.example` to `.env`.
2. Generate two different random values for `MFA_API_TOKEN` and `MFA_ENCRYPTION_KEY`. The latter must decode to exactly 32 bytes.
3. Start the container. Open the English demo, which uses one fixed disposable user named `demo`, then select Authenticator app and its TOTP tab. Scan the QR code in the setup dialog; refresh it there if needed. Code fields are masked with an eye button, and OTP fields accept digits only. No browser token is needed.

```
cp .env.example .env
openssl rand -base64 32   # MFA_API_TOKEN
openssl rand -base64 32   # MFA_ENCRYPTION_KEY
docker compose up --build -d
curl http://localhost:18083/healthz
```

3 Factor lifecycle

The factor service answers a narrow question: did the caller present a valid proof for the named account? It does not establish who the caller is. The MyScoutee backend must establish primary identity and bind the proof to that same internal user ID. Enrollment is a security-sensitive account change, not a public registration endpoint.

Both OTP methods can use an authenticator app. They use the same HMAC-based construction but choose a different moving input. TOTP derives each code from a 30-second time step; HOTP derives it from a counter. The app must explicitly support HOTP; a TOTP QR code and an HOTP QR code are not interchangeable. The detailed algorithms are defined in RFC 4226 and RFC 6238.

TOTP authenticator app

1. The service randomly generates a 20-byte shared seed, keeps it pending for ten minutes, and returns a QR image, manual secret and `otpauth://totp/` URL. Scanning stores the same seed in the user's app. A QR image is a convenient transport for the secret, not a proof of identity; anyone who copies it during setup could generate the same codes.
2. At each 30-second step, the app computes an HMAC-SHA-1 value from the seed and time counter, truncates it and displays six decimal digits. The server independently computes candidates from its encrypted copy of the seed. The digits are never the seed itself.
3. A first valid code confirms that the app received the seed. The service activates the factor, records that step as consumed and returns ten recovery codes. The user must wait for the next code before a sign-in trial.
4. For later checks, the service accepts the current step and one step either side to tolerate small clock differences. It records the highest accepted step atomically. A code from the same or an older step cannot be used again. A wider clock window would improve tolerance but increase the number of guesses that can match.

A displayed code may be accepted outside the single 30-second interval shown by the app because of the clock window; it is still rejected after the service consumes its step. The service allows at most five verification attempts per user in a 60-second window before returning 429, and clears the count after success. The first setup code is also consumed. These details matter when designing the MyScoutee login prompt and retry message.

HOTP authenticator app

HOTP uses the same 20-byte random seed, HMAC-SHA-1 and six-digit output, but its moving input is a counter. The `otpauth://hotp/` URL starts at zero. For manual setup, select HOTP, SHA-1, six digits and starting counter zero. The app entry must be explicitly counter-based: a code with a 30-second countdown is TOTP. Aegis for Android is one documented HOTP-capable app.

A valid setup code proves receipt of the seed and activates the factor. The service accepts setup counters zero through nine because an app may advance before confirmation, then stores one beyond the matched counter. Every generated app code advances its counter, even if it is never submitted.

To recover from a few skipped app codes, verification tries the service's current counter and the next nine values. On success it stores one past the accepted value in the same transaction. Earlier codes then fail, and the counter survives a restart. If the app moves beyond that ten-value look-ahead, this lab cannot verify it without a new enrollment. The ten candidates also enlarge the online guessing surface, so the attempt limit matters.

WebAuthn passkey

Registration and authentication each have *begin* and *finish* steps. `X-Ceremony-ID` identifies a five-minute, one-use ceremony. The service requests a discoverable credential and user verification. Public credential data and the authenticator counter persist in SQLite; the private key stays with the platform authenticator, synced passkey provider or compatible security key. The full challenge and origin checks are described in Section 5 below.

Recovery codes

The service generates ten independent codes, each from 16 random bytes (128 bits). They appear in a one-time popup with a Copy codes action and are cleared from the browser view when it closes. Only SHA-256 hashes are stored; a successful use deletes the matching hash. Rotation generates a fresh set and revokes all earlier codes. A passkey-only user can request the first set after registration. Removing the last active factor revokes remaining recovery codes.

A recovery code is an emergency substitute for a lost factor, not a second factor to use every day. It is deliberately much harder to guess than a six-digit OTP, but anyone who obtains an unused code can spend it. MyScoutee should show new codes only to a recently verified account holder and require an additional policy check before rotation or factor removal. The sandbox itself does not implement that application-level reauthentication.

4 Why generated codes protect an account

A six-digit output has only one million possible values. Its protection does not come from the number of digits alone. It comes from combining a secret provisioned to the user's authenticator, an input that changes over time or by counter, a separate primary login, a bounded verification window, limited online guesses and one-use handling.



Control	What it changes	What remains possible
Independent secret	A stolen password or primary session proof alone does not reveal the app's seed.	A copied QR image, compromised phone or stolen server seed can duplicate OTP generation.
Moving input	An old TOTP value ages out; an accepted HOTP value is behind the server counter.	A current code can be relayed immediately to a phishing site.
Replay state	The service consumes a TOTP time step or advances the HOTP counter after success.	It cannot undo a valid code already spent by an attacker before the real user.
Attempt limit	Five checks per user per minute cap online guessing through this API.	Availability attacks and weak application-level account binding still need separate controls.

Why this is a second factor: in an integrated login, MyScoutee would first verify its existing primary identity proof, then ask this service to verify a factor bound to the resolved internal account. A successful OTP response alone is not a login. Likewise, an API caller that can choose an arbitrary user ID and holds the global bearer token can verify or change that account's factors; the calling backend is therefore a critical trust boundary.

Security limit: real-time phishing

TOTP and HOTP are manually entered, so a fake site can ask for a fresh code and relay it to the real service before it expires. NIST explicitly does not classify manual OTP entry as phishing resistant. Rate limiting and replay prevention reduce guessing and reuse, but do not solve live relay. Prefer a passkey for phishing-resistant sign-in when the application can provide a stable HTTPS origin and a complete account policy. See NIST SP 800-63B.

MyScoutee design implication: show the factor prompt only after the Firebase proof has been checked and the backend has resolved one internal user. Associate each challenge with that login attempt. Reject a factor response for a different account, expired attempt or already authorized session. The current lab's `{"valid":true}` is a proof result, not a session-bound authorization artifact.

5 Passkey methodology

Passkey in everyday terms

A passkey is a way to prove that you control an authenticator without copying a code into the website. An authenticator can be a device's built-in credential store, a phone, a browser password manager or a FIDO2 security key. A separate password manager is not required. When you create a passkey, the authenticator generates two mathematically linked keys. It keeps the private key; this service saves the public key for the selected test user. Think of the public key as a lock the site can use to check a proof, while the private key is the only key that can produce that proof.

When you return, the site sends a fresh random challenge. You unlock the chosen authenticator with its own PIN, fingerprint or screen lock; it signs the challenge, and the site checks the signature with the saved public key. The unlock method stays with the authenticator. Some providers synchronize passkeys to your other devices; a physical security key usually keeps its credential on that key. A Google Password Manager PIN prompt belongs to Chrome's chosen provider, not to the 2FA lab, and the PIN is not sent to this service.

What the user does	Authenticator code	Passkey
Set up	Scan a QR code containing a shared seed.	Click Add passkey; choose a phone, password manager or security key offered by the browser.
Prove control later	Read and type six digits from the app.	Unlock the chosen device; it signs a fresh challenge in the background.
What the service stores	An encrypted copy of the same seed the app holds.	The public credential, never the private signing key.
Phishing behavior	A live fake page can relay a typed code.	The browser ties the credential to the site's RP ID and origin, so a lookalike domain cannot use it as that site.

WebAuthn is the browser and server protocol; a passkey is a WebAuthn credential that can provide a familiar device unlock experience. FIDO2 also covers the authenticator-side protocol used by compatible devices. These names describe layers of one ceremony, not three separate server factors. The relying party (RP) is the site named by `MFA_RP_ID` and `MFA_RP_ORIGIN`. The W3C WebAuthn specification defines the browser checks and server verification model.

How to try this lab: open `http://localhost:18083/` in a regular browser window, select a disposable user and click *Add passkey*. The VS Code built-in browser may leave the native prompt unavailable. Chrome can offer Google Password Manager, a nearby phone or a compatible security key; choose an option you can unlock now. If Chrome asks for a Google Password Manager PIN, enter that provider PIN only if you know it; do not enter a TOTP/HOTP code or your Google account password. If you do not know the PIN, cancel the prompt and

choose another available authenticator. After a successful creation the Passkeys count becomes one. *Verify passkey* then repeats a fresh challenge with the registered authenticator. This sandbox proof does not sign in to MyScoutee. See Google Chrome Help on passkeys for the provider options.

Registration: create a credential for this account

1. The already authorized backend asks the 2FA service to begin registration for the resolved user ID. The service creates a random challenge and a one-use ceremony record that expires in five minutes.
2. The browser calls `navigator.credentials.create()` with the returned options. It checks that the calling page may use the RP ID and asks an authenticator to create a key pair. This lab requests a discoverable credential and user verification, usually a device PIN, biometric unlock or security-key PIN.
3. The browser sends the attestation result and ceremony ID to the finish endpoint. The Go WebAuthn library checks the challenge, RP/origin and ceremony data, then the service stores the public credential and authenticator state under the user. The private key is never sent to the service.

Authentication: prove possession without sending a secret

1. The service begins a fresh ceremony with a new challenge and the account's allowed credentials.
2. The browser calls `navigator.credentials.get()`. After local user verification, the authenticator signs data that includes the challenge and RP context with the private key. A code is not typed into a form.
3. The finish endpoint consumes the ceremony once, checks the signed assertion against the stored public credential, and rejects a clone warning from the authenticator counter. It returns a proof result for the caller to bind to the pending login or step-up action.

The browser's origin and RP ID binding is the central phishing defense: a credential registered for the legitimate domain is not offered to an unrelated lookalike domain, and the server verifies the expected origin and challenge. A passkey may live on one security key or be synchronized across a user's devices by a provider; neither arrangement means that every device is automatically trusted by MyScoutee. The account owner still needs safe enrollment, recovery, device revocation and session policy.

MyScoutee origin and account binding

If the factor UI is later served through MyScoutee/nginx, configure its public HTTPS origin exactly in `MFA_RP_ORIGIN` and choose an RP ID valid for that domain. A passkey registered at the local `localhost` lab origin is not a production MyScoutee credential. The MyScoutee backend must mediate begin/finish for its authenticated internal account and decide whether a successful assertion completes login or authorizes a specific sensitive action.

The sandbox's BeginLogin is account-specific, not a complete username-less MyScoutee login flow. The service does not decide which MyScoutee account a browser session may claim. This is a deliberate integration boundary, especially if passkeys later become a primary sign-in method rather than a second step after Firebase identity.

6 HTTP API

Trusted backend /v1/ requests need Authorization: Bearer <MFA_API_TOKEN>. The caller maps {id} to its already authenticated user. In local demo mode only, the page issues an HTTP-only, same-site cookie scoped to /v1/ for browser trials on the configured host and fixed demo user. Public /healthz needs no token.

Method	Path after /v1/users/{id}	Result
GET	/factors	Enabled factors, passkey IDs and remaining recovery count.
POST	/totp/enrollment, /totp/confirm	Start and confirm TOTP enrollment.
POST	/hotp/enrollment, /hotp/confirm	Start and confirm HOTP enrollment.
POST	/verify	Verify totp, hotp or recovery_code.
POST	/recovery-codes/rotate	Replace and show recovery codes once.
POST	/passkeys/register/begin, /finish	Passkey registration ceremony.
POST	/passkeys/authenticate/begin, /finish	Passkey authentication ceremony.
DELETE	/passkeys/{credential-id}	Remove a passkey.
DELETE	/lab-reset for demo	Demo cookie only; delete server-side factors, codes and pending setup. Device-stored passkeys remain.

Code verification

```
POST /v1/users/demo/verify
{"method":"hotp","code":"123456"}

{"valid":true}
```

WebAuthn finish

Send the *begin* response's ceremony_id as X-Ceremony-ID. The request body is the browser's complete WebAuthn credential JSON.

Errors use a JSON error field. A request with neither a bearer token nor a valid demo cookie returns 401; a wrong bearer token also returns 401. A foreign browser origin returns 403, and a rate limit returns 429. A well-formed but wrong OTP or recovery code returns HTTP 200 with

`{"valid":false}`.

7 Docker, nginx and identity lab

The container listens on port 18083, runs without root privileges, and stores `/data/mfa.db` on a named volume. Development Compose publishes only `127.0.0.1:18083`.

Variable	Meaning
`MFA_API_TOKEN`	Required server bearer token, at least 32 characters.
`MFA_ENCRYPTION_KEY`	Required Base64 value, exactly 32 decoded bytes.
`MFA_RP_ID`, `MFA_RP_ORIGIN`	WebAuthn domain and exact browser origin; locally `localhost` and `http://localhost:18083`.
`MFA_RP_NAME`	Name displayed by the authenticator.
`MFA_DB_PATH`, `MFA_LISTEN_ADDR`	SQLite path and HTTP listening address.
`MFA_DEMO_ENABLED`	Enables the local browser page; Go default is off.

Later private deployment

The MyScoutee backend calls the private service with its server token. The browser receives only WebAuthn options and responses mediated by the application. The nginx HTTPS origin must match `MFA_RP_ORIGIN`, and its domain must match `MFA_RP_ID`. Disable the demo and loopback port publishing for an integrated deployment.

Run `docker compose -f compose.yaml -f compose.identity.yaml up --build -d` for the identity lab. Keycloak is at `127.0.0.1:18084`. OIDC and SAML have separate trial indicators. Reset clears them and uses a fresh ID token for prompt-free Keycloak logout. The SAML tab exports SP metadata XML; import it via Clients → Import client in a new realm. The bundled realm has this client. Neither result grants a MyScoutee session.

8 Data and security boundaries

The lab stores the material needed to verify future proofs, so protecting the service and its backup matters as much as protecting the browser form. At rest, the SQLite database stores OTP seeds and pending enrollments under AES-256-GCM with a random nonce per encrypted value. It stores recovery-code hashes and passkey public credential records. These controls do not make the server immune to a process compromise: a process with the encryption key can decrypt OTP seeds.

- TOTP/HOTP secrets and pending enrollments are AES-256-GCM encrypted in SQLite. Passkey private keys remain with the authenticator.
- Recovery codes are random, one-use values; only hashes are stored.
- WebAuthn ceremonies expire and are consumed once, including after a failed finish.
- Back up the database and `MFA_ENCRYPTION_KEY` together. A different key causes startup failure.
- The bearer token can be changed through the environment. Encryption-key rotation needs a data migration or factor re-enrollment.
- The SQLite deployment is designed for one service instance.

The bearer token authorizes operations for every user ID. The integrating backend must prove that each request concerns the correct user and operation. Never send the token to a production browser.

Event	Present lab control	MyScoutee responsibility
Stolen primary login proof	A separate enrolled factor can still be required.	Do not issue a fully authorized session until policy accepts the factor.
Guessed OTP	Short acceptance window, replay state and per-user attempt limit.	Protect the primary login, account lookup and API edge against abuse too.
Lost phone or security key	One-use recovery codes and factor removal endpoints.	Authenticate recovery and factor changes; notify users through existing account channels if policy requires it.
Stolen service token or database key	Encryption at rest and a server-only bearer boundary.	Keep secrets out of browsers and logs, restrict network access, rotate compromised credentials and restore safely.
Phishing site	WebAuthn origin/RP binding for passkeys; OTP has no equivalent binding.	Choose factor policy and a stable HTTPS origin; do not claim OTP is phishing resistant.

The bundled Keycloak user and admin defaults are local demonstration credentials. Real Entra ID and Duo integrations require separately configured tenants or applications. A successful external identity result alone does not authorize a MyScoutee account link or a second-factor decision.

9 MyScoutee integration cases

Current MyScoutee login uses its existing Firebase proof and `/auth/session/login` session registration. The backend also has a personal MCP OAuth authorization flow for a different delegation purpose. This sandbox changes neither.

Proposed login sequence

1 Verify Firebase proof resolve internal user → 2 Load enrolled factors apply account policy → 3 Verify chosen proof bind to pending login → 4 Create full session record factor time

This is a design path, not current behavior. The backend would need a short-lived pending login state between steps 1 and 4. The pending state should identify the same internal user, expire, and be consumed once; a factor result for another account or login attempt must not complete it. If the account has no factor, policy must explicitly decide whether ordinary login may continue or enrollment is required. The existing `/auth/session/login` currently registers a session immediately, so simply adding a 2FA call after it would leave the session boundary unchanged.

Possible use case	Later server-side connection
Second factor at login	After verifying Firebase proof and resolving the internal user, query factor state. Verify TOTP/HOTP, passkey or recovery proof before accepting or fully authorizing the application session. The current `/auth/session/login` registers a session immediately, so an intermediate state and policy decision are needed there.
Factor setup and recovery	The authenticated account settings page uses only the internal user ID resolved by the backend. Show QR provisioning and once-visible recovery codes to that account holder. Keep the service token on the server.
Step-up for sensitive actions	The backend may require a fresh factor before actions such as `/auth/me/delete` or operator configuration changes. The current 2FA API returns only `valid`; MyScoutee must bind that result to a short time window and the exact authorized action.
Enterprise SSO	The Keycloak OIDC/SAML lab demonstrates protocol exchange. Entra ID or Duo can be configured as external providers later. Map verified *issuer* + *subject* to an account through an explicit linking process; matching email alone is insufficient. Apply factor policy separately.

Account settings and recovery sequence

Enrollment should begin from an authenticated MyScoutee settings page after the backend resolves its own user ID. For TOTP/HOTP, show the QR only to that user and activate the factor only after the first code. For a passkey, run begin/finish on the correct MyScoutee origin. Show recovery codes once and ask the user to store them away from the device holding the factor. Replacing a phone, rotating codes or removing the last factor needs a recent trusted proof and an explicit recovery policy; otherwise a stolen primary session could replace the defender's factors.

Step-up and enterprise identity

For a sensitive operation such as account deletion, MyScoutee could create a pending action with an action ID, user ID and short expiry. After a factor proof, the backend marks only that pending action as approved and executes it once. A generic `valid` result must not be reusable

for unrelated operations. OIDC and SAML follow a different methodology: an identity provider authenticates the user and returns a verified identity assertion to the relying application. The local OIDC adapter uses authorization code with PKCE, state and nonce, then checks the ID token; the SAML adapter validates provider metadata, a tracked request and the signed response. Neither protocol by itself says that MyScoutee's factor policy has been met.

Keycloak is the runnable local provider in this lab. Entra ID and Duo would need their own tenant or application settings and a deliberate account mapping. For either OIDC or SAML, use a stable provider identifier and subject to link accounts; do not auto-link on email text alone. The backend then decides whether the provider's authentication context satisfies a MyScoutee policy or whether this factor service must perform a separate step. The lab does not currently consume external MFA claims as proof.

Integration boundary

The factor service verifies a proof for a user ID supplied by a trusted caller. It does not authenticate MyScoutee browser sessions, assign roles, or replace existing account management. OIDC/SAML lab results are not automatically registered as factors.

10 Verification and current limits

Go tests cover TOTP/HOTP enrollment, replay prevention, HOTP resynchronization, recovery code use and rotation, restart persistence, and signed WebAuthn registration and authentication. Negative checks cover wrong tokens, foreign browser origins, expired or disabled demo sessions, reused ceremonies, rate limits and a wrong encryption key. The English tabbed page, logo asset and QR image are checked separately.

```
go test ./...
go vet ./...
docker build -t myscoutee-2fa:local .
node --check internal/mfa/web/demo.js
```

Container checks cover `/healthz` and protected API responses. Manual trials with a real authenticator app and passkey device are still needed at the chosen integration origin. External Entra ID and Duo credentials are not bundled. This service is not yet in the MyScoutee login or session path.

There is no SMS, email or MyScoutee password database here. The optional OIDC/SAML trials are identity flows, not automatic account linking or MFA acceptance. The calling application owns the time and policy of the second-factor check.

Document ID: MSC-2FA-001 · Version: 1.5.0 · On changes, review the Go API, Compose environment and security boundaries together.

Appendix A · Technology coverage in the sandbox

The shared “2FA Stack” list combines methods, standards, client apps, identity providers and hardware. “Available” below means runnable behavior in this repository, not certification of every external product.

Items	Current role and status
TOTP Google Authenticator Microsoft Authenticator	Available. Standard TOTP provisioning URL and QR code; those apps can act as TOTP clients.
HOTP	Available. Counter-based setup and ten-step look-ahead during activation and later verification.
WebAuthn, passkeys, FIDO2 YubiKey	WebAuthn relying-party flows are available. Browser/authenticator provide the device-side FIDO2 behavior. A compatible YubiKey may work if it supports discoverable credentials and user verification; physical-device testing remains open.
OAuth 2.0, OIDC, SAML 2.0	Separate identity lab: OIDC authorization code + PKCE and SAML service provider. These are not factor types in the 2FA API.
Keycloak	Optional local Docker IdP with an imported demo realm for OIDC/SAML.
Microsoft Entra ID, Duo Security	Generic OIDC/SAML adapters can be configured, but no live tenant, credentials or verified vendor integration are bundled.
HSM	Not included. TOTP/HOTP secrets use the AES key supplied through the environment.

How to read the lab

TOTP, HOTP, passkeys and recovery codes are factor exercises. OIDC/SAML and Keycloak are external identity exercises. Recovery codes, absent from the shared list, make factor-loss recovery testable.

Appendix B · Standards and source notes

This guide distinguishes standards-level properties from behavior verified in this repository. The standards explain the algorithms and trust model; the Go source and tests determine what this lab actually implements. The design recommendations for MyScoutee are proposed integration work, not claims about a deployed feature.

- RFC 4226: HOTP - HMAC construction, counter, decimal truncation, verifier window and throttling.
- RFC 6238: TOTP - time-step moving factor and clock skew.
- W3C Web Authentication, Level 3 - RP ID, browser origin, challenge, public-key ceremony and user verification.
- NIST SP 800-63B: Authenticators - OTP phishing limitation, recovery concepts and phishing-resistant authentication.
- `internal/mfa/server.go`, `store.go`, `oidc.go`, `saml_lab.go` and tests - current lab parameters, API behavior and storage.